Group 76 Progress Report:

Elite Lu, Ishpreet Nagi, Kenneth Ong
{lue13,nagii,salimk4}@mcmaster.ca

1 Introduction

Healthcare providers deal with many vision related illnesses such as DME (Diabetic Macular Edema), CNV (Choroidal Neovascularization), and drusen that impair vision. They utilize retinal scans to detect and determine the conditions. An issue that arises is the uniqueness of each ailment, as each condition varies in subtle ways. Correctly identifying retinal diseases can prove to be a challenge in a plethora of cases and with approximately 30 million OCT scans performed each year, this process can prove time-consuming for healthcare professionals. By having a classification model that can help speed up this process, we can allow healthcare professionals to focus their attention on helping more individuals requiring their aid.

2 Related Work

Here, talk about the related work you encountered for your approach. Cite at least 5 references. Refer to item 2. No one has done exactly your task? Write about the most similar thing you can find. This should be around 0.25-0.5 pages.

3 Dataset

3.1 Folder Structure

When downloading the data, the dataset was partitioned as follows:

```
Dataset
├── test
│   ├── CNV
│   ├── DME
│   ├── DRUSEN
│   └── NORMAL
├── train
│   ├── CNV
│   ├── DME
│   ├── DRUSEN
│   └── NORMAL
└── val
    ├── CNV
    ├── DME
    ├── DRUSEN
    └── NORMAL
```

3.2 Preprocessing

Each image was loaded from the various folders as a PIL (Pillow, not the Python Imaging Library) Image and then converted to a tensor object. The pipeline varies slightly between *train* when compared to both *test* and *val* (abbreviated to *TV* pipeline for brevity). Both pipelines resized the images to 224 pixels by 224 pixels and normalized it with means [0.485, 0.456, 0.406] and standard deviation (std) of [0.229, 0.224, 0.225]. The values are the means and standard deviations for ImageNet, which is the dataset that ResNet is being trained on. The only difference is that before normalization, the *train* dataset images might be flipped whereas the image data for the *TV* pipeline is not. This allowed for the training set to have a more robust and diverse dataset. However, this is technically not considered an augmentation since we did not create new data (no data distribution changes).

3.3 Annotation

Each image is labeled with an integer to represent its corresponding disease. The images were already named as DISEASE-#####-#.jpeg and located

in their corresponding folders, which allowed for ease of labeling. The numbers for their corresponding disease is shown below:

CNV	0
DME	1
DRUSEN	2
NORMAL	3

4 Features

4.1 Data Input

Since we are doing a computer vision/image classification task, the feature were derived from the pixels of the images. The input for the model was images, which are comprised of pixels. Each pixel is denoted by three values: Red(R), Green(G), and Blue(B). As mentioned previously, each image was cropped and normalized with values corresponding the means and standard deviations for images from ImageNet.

4.2 Features

In terms of selecting the in features, we did not directly do that. This is because it is not required as there are no features to select. Since the pixels are the "feature", this would not be justified. As a result, feature engineering and feature selection was not needed. The same can be said for learning embeddings, as this is not a natural language processing task.
For the out features, there are four main features, which are the four categories of dieases: CNV, DME, DRUSEN, and NORMAL.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

5.1 Learning Model

Instead of training a model from scratch, we used a pretrained neural network and adapted it to our use case. The pretrained neural network selected was ResNet-50, which is trained on the ImageNet dataset. ResNet-50 is a neural network that is 50 layers deep originally developed by Microsoft Research in 2015. This is a type of convolutional neural network (CNN), which is a subset of neural networks that are used for image recognition.

5.2 Implementation Steps

5.2.1 Initializing the Neural Network

To implement the ResNet-50 based neural network, the first step was to instantiate a neural network that uses ResNet-50. This occurs after all the images have been preprocessed. This was called CustomResNet. For the initialization, the input features and classifiers are all defined. The initialization is very similar to typical setup for neural networks.

5.2.2 Freezing and Unfreezing Layers

Upon initialization, the first thing is to freeze all the layers but the last layer, the classifier layer. This is because a pretrained model such as ResNet 50 already comes with various useful general features such as edges, textures, and colours in early layers. Since the dataset we use is quite small (four disease classes), training all the layers can result in an increase of time, overfitting, and wiping out pretrained knowledge.

5.2.3 Defining the Criterion and Optimizer

Before compiling the model, two additional items need to be addressed, which are the criterion and optimizer. For the criterion, we used Cross Entropy from the torch.nn (neural network) library. For the optimizer, we used Adam (Adaptive Moment Estimation) from the torch.optim library. The Adam stochastic optimization method uses two key concepts, which are Momentum and Root Mean Square Propagation (RMSprop), combining them to balance the optimization process. The learning rate defined in Adam is 0.001, which is common for machine learning. The optimization is performed only on the parameters of the final layer since the other layers are currently frozen.

5.2.4 Training Data

The dataset has 83484 images in the four folders (CNV, DME, DRUSEN, NORMAL) under training. The distribution is not even, as CNV had significantly more images compared to the other diseases. For this milestone, we took 1000 random photos from each folder of the dataset, resulting in an even spread for each disease. This totaled 4000 images for model training for this milestone. We plan on using all the photos and additional augmentation for the categories with fewer images in the final model to prevent skewing and bias.

5.2.5 Validation and Test Data

Since there are significantly fewer images categorized under these, we used all of them. There are only 32 images for validation spread evenly in the four diseases. The testing data has an even spread too, with 242 images for each disease, totaling 968 images.

5.2.6 Training and Validation

For training, we used 10 epochs for the 4000 selected images. The loss and accuracy of both training and validation were printed along with a graph displaying the loss for training and validation.

5.2.7 Testing

After testing, the values were plotted on a 4×4 confusion matrix. The final accuracy of the model is written above it.

6 Results and Evaluation

As previously stated, the found dataset was already organized neatly, with the data split into train, validation, and test folders. Following this preset procedure, during the preprocessing stage, we split the data into train/validation/test splits, with the composition of each being: 1000 images for training, 32 images for validation, and 968 images for testing. As a reminder, the training images were limited to only 1000 images at this stage to ensure fast computation, allowing for quicker model tuning. Since our dataset is so rich with images, we will not be opting to use cross-validation to allow for faster computation.

After the model is initialized, it is fed through the three stages of training, validation, and testing, with the training and validation phases being combined in a singular loop to save computation time as well as allow for better performance metric representation. During the training and validation phase, the model's performance is measured via the computation of training/validation loss and training/validation accuracy at each epoch. This allows us insight into the performance trend of the model and its overall journey through the training and validation phase, as we can witness the rate at which the model learns the trends in the dataset and becomes better at classifying the images through the loss and accuracy metrics. Our loss method is the cross-entropy loss, which we access through the nn.CrossEntropyLoss() function that preexists as a part of PyTorch, and the accuracy is calculated via the division of the correct predictions by the model on both the training and validation datasets by the total predictions by the model on the training and validation sets, respectively. We are also storing the loss values calculated for the training and validation steps, and then visualizing the overall trends by plotting them on a line graph to help us further understand the model and its performance.

By this stage, the model has been evaluated once through the validation phase; however, it is evaluated once again, this time on the test split of the dataset via the testing phase. For the testing phase, we are simply using the performance metric of accuracy as our evaluation metric of the performance of the model, which is calculated via the same accuracy calculation method as training and validation. In addition to these performance metrics, we are also outputting a confusion matrix that visualizes the true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions of the model for the testing phase. This helps us better evaluate the performance of the model on the testing dataset, thus allowing us to better understand the ways to improve it.

So far, we have the following results, which will be acting as our baselines going forward. Results will be shown in the next page.

```
--------------------------------------------------------------------------------
----------------------- Training and Validation Phase ----------------------
--------------------------------------------------------------------------------

Epoch [1/10]:
Training - Train Loss: 1.0887, Train Accuracy: 0.5433
Validation - Val Loss: 0.6531, Val Accuracy: 0.7812
Epoch [2/10]:
Training - Train Loss: 0.7874, Train Accuracy: 0.7192
Validation - Val Loss: 0.5057, Val Accuracy: 0.8750
Epoch [3/10]:
Training - Train Loss: 0.6839, Train Accuracy: 0.7508
Validation - Val Loss: 0.4508, Val Accuracy: 0.8438
Epoch [4/10]:
Training - Train Loss: 0.6904, Train Accuracy: 0.7365
Validation - Val Loss: 0.3884, Val Accuracy: 0.9062
Epoch [5/10]:
Training - Train Loss: 0.6213, Train Accuracy: 0.7712
Validation - Val Loss: 0.3650, Val Accuracy: 0.9375
Epoch [6/10]:
Training - Train Loss: 0.6065, Train Accuracy: 0.7668
Validation - Val Loss: 0.3354, Val Accuracy: 0.9062
Epoch [7/10]:
Training - Train Loss: 0.5787, Train Accuracy: 0.7800
Validation - Val Loss: 0.3003, Val Accuracy: 0.9062
Epoch [8/10]:
Training - Train Loss: 0.5600, Train Accuracy: 0.7983
Validation - Val Loss: 0.3019, Val Accuracy: 0.9062
Epoch [9/10]:
Training - Train Loss: 0.5657, Train Accuracy: 0.7867
Validation - Val Loss: 0.3093, Val Accuracy: 0.9062
Epoch [10/10]:
Training - Train Loss: 0.5466, Train Accuracy: 0.7920
Validation - Val Loss: 0.2906, Val Accuracy: 0.9062
```

Figure 1: Training and validation loss over epochs.

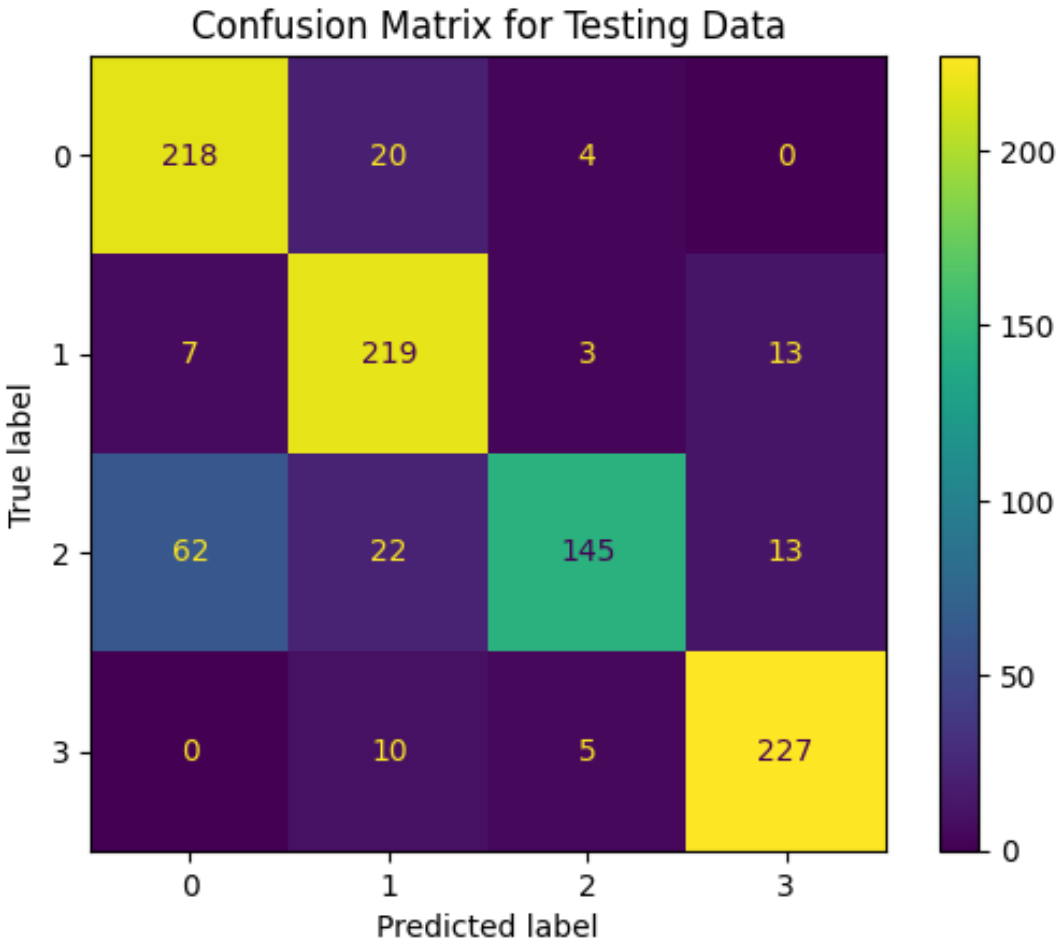

Figure 2: Confusion Matrix of the testing results.

As one can clearly see in the results above, while our model is on the right track, there is a large margin for improvement.

7 Feedback and Plans

Looking at the results of our evaluation and metrics, we believe that we are on the right track. It seems that we had successfully achieved an adequate model baseline that is trained on a small subset of the dataset.

7.1 Image Processing and Augmentation

As mentioned in section 3, we did some image preprocessing that involved cropping, horizontally flipping, and normalizing the images. This is the first point of improvement that we think we could make. From our discussion with Parisa, we identified that the distribution of our training data is skewed towards CNV and Normal, which may cause overfitting and bias towards those labels. So, we could augment the training data for DME and Drusen so that there is about the same number of data for each category. Building upon this, we could also add more image transformations such as adding jitter and affine transformations to introduce imperfections in the image data, which could help make the model more robust through adversarial training.

7.2 Additional Metrics

The second point of improvement is that we could add more metrics to help us evaluate our results better. Parisa mentioned that it might be useful to have precision, recall, and F-1 score as a metric in addition to the confusion matrix and accuracy that we currently have. Additionally, we believe that it might be worthwhile to also have the area under the ROC curve (AUC) as a metric for accuracy.

7.3 Hyperparameters and Dataset

Once we have implemented the improvements outlined above, our plan would be to train the model on a bigger subset of the training dataset (around 5000 data points) to find the best hyperparameters for the model. While training, we are aiming to optimize for the recall and area under the ROC curve for this project. This is because we want our classification model to minimize the number of false negatives, which in this case means that patients that actually have the disease should not go undiagnosed.

Finally, we would do a final training run using the best hyperparameters on the entire training and validation dataset and then evaluate them using our testing dataset at the end.

8 Team Contributions

8.1 Elite

- Worked on the initial image opening and storing of data.
- Wrote sections 1, 3, 4, and 5 of the document.

8.2 Ishpreet

- Trained and tested the model.
- Wrote section 6 of the document.

8.3 Kenneth

- Worked on setting up the neural network and research for the start of the project.
- Wrote sections 2 and 7 of the document.